Task 9 - Implement and Test the Billing Program

C Programming Assignment - Rural Electricity Billing System

Tariff assumption

The screenshot does not show the complete official tariff table. The demonstration program uses illustrative rates: first 100 units at 150 paise/unit, next 100 at 230 paise/unit, remaining units at 300 paise/unit, plus fixed charge ■50. Replace these with the official tariff before submission.

Manual tests

0 units = ■50.00; 100 units = ■200.00; 200 units = ■430.00; 250 units = ■580.00; 350 units = ■880.00.

Required edge cases

Test exactly 100 and 200 units, zero usage, invalid text, negative readings, current below previous, confirmed rollover and bills with manually verified totals.

Rollover

For confirmed 16-bit rollover, consumed = (65535 - previous) + 1 + current. Previous 65530 and current 5 gives 11. Use only if the meter specification confirms rollover.

Code note

A complete compilable C demonstration is included as billing.c inside the ZIP. It validates readings, rejects current readings below previous readings, calculates slab charges in integer paise and prints the total.

C program

```c
/* Illustrative tariff only: replace with official rates. */
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>

#define RATE1_PAISE 150LL
#define RATE2_PAISE 230LL
#define RATE3_PAISE 300LL
#define FIXED_CHARGE_PAISE 5000LL

static int read_u64(const char *prompt, uint64_t *value) {
    char line[128], *end;
    unsigned long long n;
    printf("%s", prompt);
    if (!fgets(line, sizeof line, stdin)) return 0;
    errno = 0;
    if (line[0] == '-' || line[0] == '\n') return 0;
    n = strtoull(line, &end, 10);
    if (errno == ERANGE || end == line) return 0;
    while (*end == ' ' || *end == '\t' || *end == '\n' || *end == '\r') ++end;
    if (*end != '\0') return 0;
    *value = (uint64_t)n;
    return 1;
}

static int bill(uint64_t units, int64_t *total) {
    uint64_t a = units < 100 ? units : 100;
    uint64_t b = units > 100 ? (units - 100 < 100 ? units - 100 : 100) : 0;
    uint64_t c = units > 200 ? units - 200 : 0;
    if (a > (uint64_t)INT64_MAX / RATE1_PAISE ||
        b > (uint64_t)INT64_MAX / RATE2_PAISE ||
        c > (uint64_t)INT64_MAX / RATE3_PAISE) return 0;
    int64_t energy = (int64_t)a * RATE1_PAISE
                   + (int64_t)b * RATE2_PAISE
                   + (int64_t)c * RATE3_PAISE;
    if (energy > INT64_MAX - FIXED_CHARGE_PAISE) return 0;
    *total = energy + FIXED_CHARGE_PAISE;
    return 1;
}

int main(void) {
    uint64_t previous, current, units;
    int64_t total;
    if (!read_u64("Previous reading: ", &previous) ||
        !read_u64("Current reading: ", &current)) {
        fprintf(stderr, "Invalid input. Enter non-negative whole numbers.\n");
        return EXIT_FAILURE;
    }
    if (current < previous) {
        fprintf(stderr, "Current reading is below previous reading; verify rollover.\n");
        return EXIT_FAILURE;
    }
    units = current - previous;
    if (!bill(units, &total)) {
        fprintf(stderr, "Bill exceeds supported range.\n");
        return EXIT_FAILURE;
    }
    printf("Units consumed: %" PRIu64 "\n", units);
```

```
    printf("Illustrative bill: INR %" PRId64 ".%02" PRId64 "\n",
           total / 100, total % 100);
    return EXIT_SUCCESS;
}
```

Complexity

For S slabs, billing is O(S) time and O(1) extra space. A fixed three-slab tariff is O(1).

Important: illustrative tariff rates are used because the screenshot does not show the complete official tariff table. Replace them with your instructor's prescribed rates.